

m2424: математическая модель библиотеки

1. Исходные объекты

Библиотека работает с вещественным или комплексным вектором

$$x = (x_0, \dots, x_{m-1}) \in \mathbb{C}^m.$$

СККС-профиль задаётся набором

$$P = (N, Q, \Delta, S),$$

где

$$N \in 2\mathbb{N}, \quad Q = \prod_{i=0}^L q_i, \quad \Delta > 0, \quad S = \frac{N}{2}.$$

Здесь N — степень полиномиального модуля,

$$R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$$

— кольцо ciphertext, Q — цепочка коэффициентных модулей, Δ — масштаб СККС, S — число СККС-слотов.

Для bootstrapping-профиля в текущей реализации используется

$$N = 16384, \quad S = 8192, \quad \Delta = 2^{40},$$
$$\log_2(q_0), \dots, \log_2(q_L) = 60, 40, 40, 40, 40, 40, 40, 40, 60.$$

2. Кодирование, шифрование и состояние ciphertext

Кодирование СККС переводит вектор x в plaintext

$$p = \text{Encode}_\Delta(x) \in R_Q.$$

Шифрование задаёт ciphertext

$$c = \text{Enc}_{pk}(p) \in R_Q^2.$$

Расшифрование и декодирование дают приближённый результат

$$\hat{x} = \text{Decode}_\Delta(\text{Dec}_{sk}(c)).$$

Состояние ciphertext описывается тройкой

$$\text{state}(c) = (\text{level}(c), \text{scale}(c), \text{size}(c)),$$

где $\text{level}(c)$ задаёт текущую позицию в modulus chain, $\text{scale}(c)$ задаёт текущий СККС-масштаб, а $\text{size}(c)$ задаёт число полиномов ciphertext.

3. Базовые гомоморфные операции

Для ciphertext $c_x = \text{Enc}(x)$ и $c_y = \text{Enc}(y)$ библиотека реализует следующие операции.

Сложение:

$$c_{x+y} = c_x + c_y, \quad \text{Dec}(c_{x+y}) \approx x + y.$$

Вычитание:

$$c_{x-y} = c_x - c_y, \quad \text{Dec}(c_{x-y}) \approx x - y.$$

Умножение ciphertext на plaintext-вектор a :

$$c_{a \odot x} = \text{Rescale}(c_x \cdot \text{Encode}(a)), \\ \text{Dec}(c_{a \odot x}) \approx a \odot x.$$

Умножение двух ciphertext:

$$c_{xy} = \text{Rescale}(\text{Relin}_{rlk}(c_x \cdot c_y)), \\ \text{Dec}(c_{xy}) \approx x \odot y.$$

После умножения масштаб увеличивается:

$$\Delta \cdot \Delta = \Delta^2.$$

Операция Rescale делит ciphertext на следующий модуль q_i и возвращает масштаб к величине порядка Δ :

$$\text{scale}(\text{Rescale}(c)) \approx \frac{\text{scale}(c)}{q_i}.$$

При этом уровень ciphertext уменьшается на один.

Ротация слотов:

$$\text{Rot}_k(c_x) = \text{Enc}(x^{(k)}),$$

где

$$x_i^{(k)} = x_{(i+k) \bmod S}.$$

4. Метрики ошибки

Для точного вектора x и результата $\hat{x}$ используются метрики

$$e_{\max}(x, \hat{x}) = \max_i |x_i - \hat{x}_i|,$$

$$e_{\text{mean}}(x, \hat{x}) = \frac{1}{m} \sum_{i=0}^{m-1} |x_i - \hat{x}_i|.$$

Критерий прохождения вычисления:

$$\text{ok}(x, \hat{x}, \tau) = \begin{cases} 1, & e_{\max}(x, \hat{x}) \leq \tau, \\ 0, & e_{\max}(x, \hat{x}) > \tau. \end{cases}$$

Для bootstrapping-harness используется

$$\tau = 2 \cdot 10^{-5}.$$

5. ABFT-инварианты

Для полезной нагрузки

$$x = (x_0, \dots, x_{m-1})$$

задаётся checksum

$$\sigma(x) = \sum_{i=0}^{m-1} x_i.$$

Расширенный вектор:

$$x^+ = (x_0, \dots, x_{m-1}, \sigma(x)).$$

Для сложения:

$$\sigma(x + y) = \sigma(x) + \sigma(y).$$

Для вычитания:

$$\sigma(x - y) = \sigma(x) - \sigma(y).$$

Для ротации:

$$\sigma(x^{(k)}) = \sigma(x).$$

Для покомпонентного умножения проверяется равенство наблюдаемой суммы CPU-эталону:

$$\sigma(x \odot y) = \sum_{i=0}^{m-1} x_i y_i.$$

ABFT-проверка задаётся условием

$$|\sigma_{\text{observed}} - \sigma_{\text{expected}}| \leq \tau_{\text{abft}}.$$

6. Линейные преобразования через ротации

Пусть

$$D_k = \text{diag}(d_{k,0}, \dots, d_{k,S-1})$$

— диагональная plaintext-матрица, а Rot_k — оператор ротации СККС-слов. Тогда линейное преобразование задаётся как

$$L(c) = \sum_{k \in K} D_k \text{Rot}_k(c).$$

В расшифрованном виде:

$$\text{Dec}(L(c_x)) \approx \sum_{k \in K} D_k x^{(k)}.$$

Для плотных диагональных матриц используется baby-step/giant-step. Пусть

$$b = \left\lceil \sqrt{|K|} \right\rceil, \quad k = i + jb, \quad 0 \leq i < b.$$

Baby-steps:

$$B_i = \text{Rot}_i(c).$$

Giant-группа:

$$U_j = \sum_i D_{i,j} B_i.$$

Итоговое преобразование:

$$L(c) = \sum_j \text{Rot}_{jb}(U_j).$$

Для dense-transform на 16 логических slots число rotation keys снижается с 15 до 6.

7. Сумма слотов

Для $m = 2^d$ сумма первых m слотов вычисляется последовательностью

$$c^{(0)} = c,$$

$$c^{(j+1)} = c^{(j)} + \text{Rot}_{2^j}(c^{(j)}), \quad j = 0, \dots, d-1.$$

После d шагов первый слот содержит

$$(\text{Dec}(c^{(d)}))_0 \approx \sum_{i=0}^{m-1} x_i.$$

8. Полиномиальные вычисления

Для полинома

$$f(t) = \sum_{j=0}^r a_j t^j$$

вычисление над ciphertext задаётся как

$$f(c) = \sum_{j=0}^r a_j c^j,$$

где произведения выполняются через

MulRelinRescale.

Каждое ciphertext-ciphertext умножение расходует один уровень modulus chain. Перед сложением членов полинома выполняется выравнивание уровней и масштабов.

9. EvalMod

В bootstrapping-harness используется нечётный полином степени 7:

$$P_7(u) = u - 6.579736267393u^3 + 12.98787880453u^5 - 12.20811674381u^7.$$

Рабочий интервал:

$$|u| \leq 2^{-10}.$$

CPU-форма:

$$y = P_7(u).$$

Ciphertext-форма:

$$\begin{aligned} c_{u^2} &= c_u^2, & c_{u^3} &= c_u \cdot c_{u^2}, \\ c_{u^5} &= c_{u^3} \cdot c_{u^2}, & c_{u^7} &= c_{u^5} \cdot c_{u^2}. \end{aligned}$$

После выравнивания уровней:

$$P_7(c_u) = c_u - 6.579736267393c_{u^3} + 12.98787880453c_{u^5} - 12.20811674381c_{u^7}.$$

10. CoeffToSlot и SlotToCoeff

Пусть M — матрица канонического вложения СККС для заданного числа логических slots:

$$z = Mx.$$

В реализации матрица строится из корней единицы:

$$M_{ij} = \zeta^{g_i j}, \quad \zeta = \exp\left(\frac{2\pi i}{4n}\right), \quad g_i = 5^i \bmod 4n.$$

Преобразование CoeffToSlot:

$$\text{CoeffToSlot}(c_x) = L_M(c_x),$$

где L_M — diagonal-transform матрицы M .

Преобразование SlotToCoeff:

$$\text{SlotToCoeff}(c_z) = L_{M^{-1}}(c_z).$$

CPU-критерий обратимости:

$$M^{-1}Mx = x.$$

Ciphertext-критерий:

$$\text{Dec}(\text{SlotToCoeff}(\text{CoeffToSlot}(c_x))) \approx x.$$

11. Refresh-harness

Вход:

$$x \in \mathbb{C}^m, \quad m = 16$$

для текущего демонстрационного refresh-harness.

Упаковка во все СККС-слоты:

$$\tilde{x}_i = x_{i \bmod m}, \quad i = 0, \dots, S-1.$$

Шифрование:

$$c_0 = \text{Enc}(\text{Encode}_\Delta(\tilde{x})).$$

Этап 1. Подготовка уровня:

$$c_1 = \text{ModRaiseHarness}(c_0).$$

Этап 2. Переход в slot-представление:

$$c_2 = \text{CoeffToSlot}(c_1).$$

Этап 3. Нормализация:

$$\alpha = \begin{cases} 1, & \max_i |z_i| \leq 2^{-11}, \\ \frac{2^{-11}}{\max_i |z_i|}, & \max_i |z_i| > 2^{-11}, \end{cases} \quad z = Mx,$$

$$c_3 = \alpha c_2$$

через управляемое изменение CKKS scale без расхода уровня modulus chain.

Этап 4. Полиномиальная модульная редукция:

$$c_4 = P_7(c_3).$$

Этап 5. Возврат в coefficient-представление:

$$c_5 = \text{SlotToCoeff}(c_4).$$

Этап 6. Обратная нормализация:

$$c_6 = \alpha^{-1} c_5.$$

Этап 7. Восстановление первой RNS-базы:

$$c_7 = \text{ModRaise}(c_6).$$

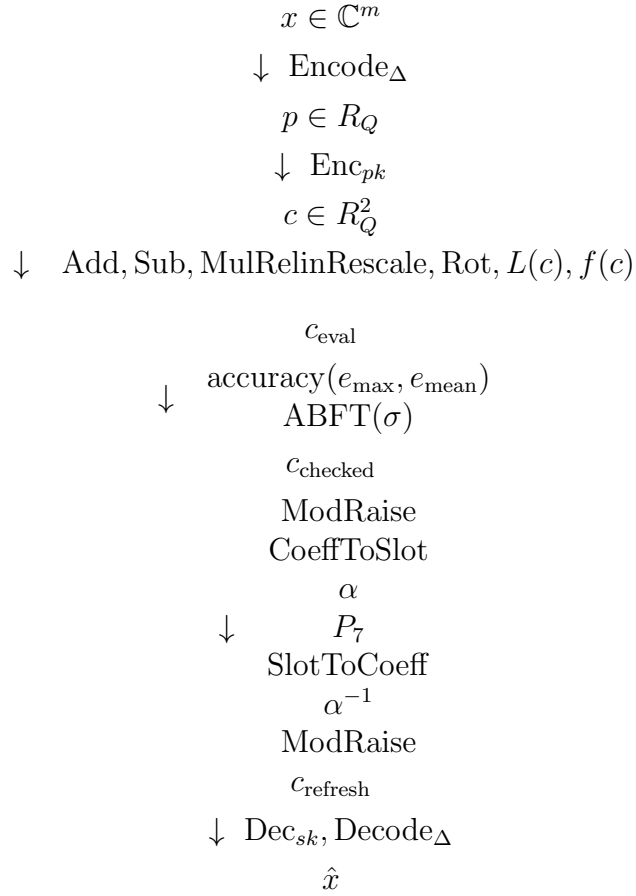
Итоговый критерий сохранения значения:

$$e_{\max}(x, \text{head}_m(\text{Decode}(\text{Dec}(c_7)))) \leq 2 \cdot 10^{-5}.$$

Критерий доступности результата:

$$\text{level}(c_7) > \text{level}(c_5).$$

12. Общая схема вычисления



13. Формальный результат

Библиотека реализует отображение

$$\mathcal{F}_{m2424} : (x, P, \tau) \mapsto (c_{\text{out}}, \hat{x}, R),$$

где c_{out} — результирующий ciphertext, $\hat{x}$ — расшифрованный приближённый результат, R — отчёт о вычислении.

Отчёт имеет вид

$$R = (e_{\text{max}}, e_{\text{mean}}, \text{level}, \text{scale}, \text{status}).$$

Статус вычисления:

$$\text{status} = \begin{cases} \text{PASS}, & e_{\text{max}} \leq \tau, \\ \text{FAIL}, & e_{\text{max}} > \tau. \end{cases}$$

Для refresh-harness текущий результат удовлетворяет

$$e_{\text{max}} \approx 1.36 \cdot 10^{-5}, \quad \tau = 2 \cdot 10^{-5}, \quad \text{status} = \text{PASS}.$$